Introduction of CSS

CSS stands for **Cascading Style Sheets**.
It is a **styling language** used to describe
how HTML elements should look on a web page.
The word **cascading** refers to how **multiple CSS rules** are applied
to the same element when there are conflicts.

What is CSS?

Definition: Cascading Style Sheets (CSS) is a stylesheet language used to describe the presentation of a document written in HTML.

Why CSS?

Separation of Concerns: Keeps HTML clean by separating content from design.
Reusability: Enables consistent styling across multiple pages using one stylesheet.
Improved User Experience: Offers visually appealing and responsive designs.

History and Evolution:

CSS1 (1996): Introduced basic styling capabilities.
CSS2 (1998): Added media types, positioning, and table layouts.
CSS3 (1999-Present): Modular approach, animations, transitions, and responsive design.

Structure of a CSS Rule:

```
selector {
    property: value;
}
```

Selector: Targets the HTML element to style (e.g., h1, .class, #id).
Property: Defines the style attribute (e.g., color, font-size).
Value: Specifies the style (e.g., red, 24px).

Example

```
h1 {
    color: blue;
    font-size: 24px;
}
```

CSS Comments:

- Comments are ignored by browsers and help document your code.

```
/* This is a comment */
```

Best Practices:

- Use meaningful and readable selectors.
- Avoid using IDs for styling (prefer classes).
- Keep styles modular and reusable.

Ways to Add CSS

- Inline, Internal and External CSS.

Inline CSS:

- Applies styles directly within an HTML element.

Example: <h1 style="color: red;">Hello World</h1>

Pros: Quick and specific.
Cons: Difficult to maintain and reuse.

Internal CSS:

- Defined within the <style> tag in the <head> section of an HTML document.

```
<style>
    h1 {
      color: red;
        }
</style>
```

Pros: Centralized for single-page styling.
Cons: Not reusable across multiple pages.

External CSS:

- Linked via an external file using the <link> tag.

Example: <link rel="stylesheet" href="styles.css">

Pros: Ideal for large projects, reusable across multiple pages.

Task 1: Create a simple webpage with inline, internal, and external CSS demonstrating different styling methods.

CSS Selectors

What Are Selectors?

- Selectors are used to "select" the elements you want to style.

Types of Selectors:

Universal Selector: Targets all elements (*)
Type Selector: Targets specific HTML tags (h1, p)
Class Selector: Targets elements with a specific class (.class)
ID Selector: Targets elements with a specific ID (#idname)

Grouping Selectors:

- Combine multiple selectors for shared styles.

```
h1, p {
   color: green;
}
```

What are CSS Properties?

- CSS properties are rules used to style and layout HTML elements.

Why Use CSS Properties?

- Enhance aesthetics.
- Improve readability and accessibility.
- Control layouts for different devices.

CSS Transitions & Animations

- Transitions: Allow smooth changes between states.

Example:

```
.button:hover {
  background-color: red;
  transition: background-color 0.5s;
}
```

- Animations: Define keyframes for custom animations.

Example:

```
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

.box {
  animation: fadeIn 2s ease-in-out;
}
```

CSS Pseudo-classes & Pseudo-elements

Pseudo-classes

- Define a special state of an element.

Common examples:

- :hover - Applies styling when the mouse is over an element.
- :focus - Styles an element when it is selected.
- :nth-child(n) - Targets specific child elements.

Example:

```
button:hover {
    background-color: blue;
}

input:focus {
   border: 2px solid green;
}
```

Pseudo-elements

- Style specific parts of an element.

Common examples:

- ::first-letter - Styles the first letter of text.
- ::first-line - Styles the first line of text.
- ::before & ::after - Insert content before/after an element.

Example:

```
p::first-letter {
   font-size: 2em;  color: red;
}
p::before {
  content: "★ ";  color: gold;
}
```

Difference:

- Pseudo-classes affect the whole element based on a condition.
- Pseudo-elements style a specific part of an element.

Text Styling

Key Text Properties

- Color

Example: color: red;

- Font-Family

Example: font-family: Arial, sans-serif;

- Font-Size

Example: font-size: 16px;

- Text-Alignment

Example: text-align: center;

Example Code:

```
p {
  color: blue;
  font-family: "Verdana", sans-serif;
  font-size: 18px;
  text-align: justify;
}
```

Background Properties

- Background-Color

Example: background-color: lightgrey;

- Background-Image

Example: background-image: url('image.jpg’);

- Background-Position

Example: background-position: center;

Example Code:

```
body {
 background-color: #f0f0f0;
 background-image: url('pattern.png');
 background-position: top left;
}
```

Borders

Border Properties

- Width

Example: border-width: 2px;

- Style

Example: border-style: solid;

- Color

Example: border-color: black;

- Border-Radius

Example: border-radius: 10px;

Example Code:

```
div {
  border-width: 3px;
  border-style: dotted;
  border-color: green;
  border-radius: 15px;
}
```

Margins and Padding

Key Differences

Margins: Space outside an element.
Example: margin: 20px;

Padding: Space inside an element, around content.
Example: padding: 15px;

Example Code:

```
div {
  margin: 30px;
  padding: 20px;
}
```

Introduction to Responsive Design

What is Responsive Design?

- Adapts the layout to different screen sizes and devices.
- Ensures usability and aesthetics across platforms.

Media Queries

- Basics of Media Queries

Syntax:

```
@media (max-width: 600px) {
    body {
      background-color: lightblue;
    }
}
```

Usage: Adjust styles for specific screen sizes.

Example Code:

```
@media (min-width: 768px) {
    p {
      font-size: 18px;
    }
}
```

Understanding Flexbox

What is Flexbox?

- A CSS layout module designed for efficient alignment and distribution of elements within a container.

Key features: Flexibility, responsiveness, and alignment control.

Core Concepts

Flex Container: The parent element.
Use display: flex; to activate Flexbox.

Flex Items: The children of the flex container.

Flexbox Properties

For the container:

- flex-direction: Defines the direction of items (row, column).
- justify-content: Aligns items along the main axis(left to right).
- align-items: Aligns items along the cross axis.
- flex-wrap: Allows items to wrap onto multiple lines.

For the items:

- flex-grow, flex-shrink, and flex-basis: Control how items grow, shrink, or occupy space.

Practical Examples

- A navigation bar layout.
- A responsive card grid.

Alignment in Flexbox

- justify-content: Align items horizontally.

Options: flex-start, center, space-between, space-around, etc.

- align-items: Align items vertically.

Options: stretch, flex-start, center, etc.

Distribution of Space

Using properties like gap (newer CSS feature) to add consistent spacing between elements.

Examples

1. Centering a login form in the middle of a page.
2. Distributing items evenly in a gallery layout.

For Dynamic Images:

https://i.pravatar.cc/300

Custom Fonts and Icons:

To add custom fonts and icons you can import the fonts and icons from CDN link

CSS Positioning

Position Property Values:

Static (Default): Elements/tag follow the natural flow of the document.
Relative: Positioned relative to its normal position.
Absolute: Positioned relative to the nearest positioned ancestor.
Fixed: Positioned relative to the viewport, does not scroll.
Sticky: Toggles between relative and fixed based on scroll position.

Z-Index:

Controls the stacking order of elements (higher values are on top).

Examples:

```
.box {
    position: absolute;
    top: 50px;
    left: 100px;
  }

 .sticky {
     position: sticky;
     top: 0;
     background-color: yellow;
    }
```

Task 2: Create a webpage with a fixed navbar and content sections demonstrating various positioning properties.

Create a Personal Portfolio Webpage

Title: Task 1 - Personal Portfolio

Content:

Objective: Build a simple portfolio webpage.

Requirements:
Header with name & navigation.

Sections: About, Projects, Contact.

- Apply background color, font styles, margins, and padding.
- Add hover effects to links.

Expected Outcome: A basic, styled personal portfolio page.

Design a Simple Product Card

Title: Task 2 - Product Card

Content:

Objective: Create a product display card.

Requirements:
Add product image, name, price, and a button.

Style with CSS:
- Border and shadow for card.
- Centered text.
- Button with hover effect.
- Expected Outcome: A clean, visually appealing product card.

CSS Frameworks and Libraries

What Are CSS Frameworks?

Frameworks provide pre-written CSS rules and components to simplify development.

Popular Frameworks:

Bootstrap: Offers a grid system, responsive utilities, and UI components.
Tailwind CSS: A utility-first framework with pre-defined classes.

Advantages:

- Speeds up development.
- Ensures consistent designs.
- Predefined components reduce the need for custom CSS.

CSS Best Practices

- Performance optimization - Use efficient selectors.
- Maintainability - Use meaningful class names.
- Avoid unnecessary overrides - Keep styles structured.
- Minification & Compression - Reduce file size for faster loading.

Final Project

Project: Create a responsive webpage for a fictional product or service using HTML and CSS.

Requirements:

- Include a header, navbar, main content, and footer.
- Use external CSS and responsive design principles.
- Apply at least one CSS framework.
- Use advanced positioning and include animations or hover effects.

Submission Notes

Capture Evidence:

- Take screenshots or record a short video for each task.
- Ensure your evidence clearly shows the completed task.

Upload Tasks:

- Upload each task as a separate folder or file in your GitHub repository.

LinkedIn Post:

- Create a short post summarizing what you learned and how you completed the tasks.
- Highlight your key takeaways.
- Mention the following account in your post:
- **Shahzad khan**

Thank You